LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Tofa Suwarna 2609106037

Kelas B1 ‘26

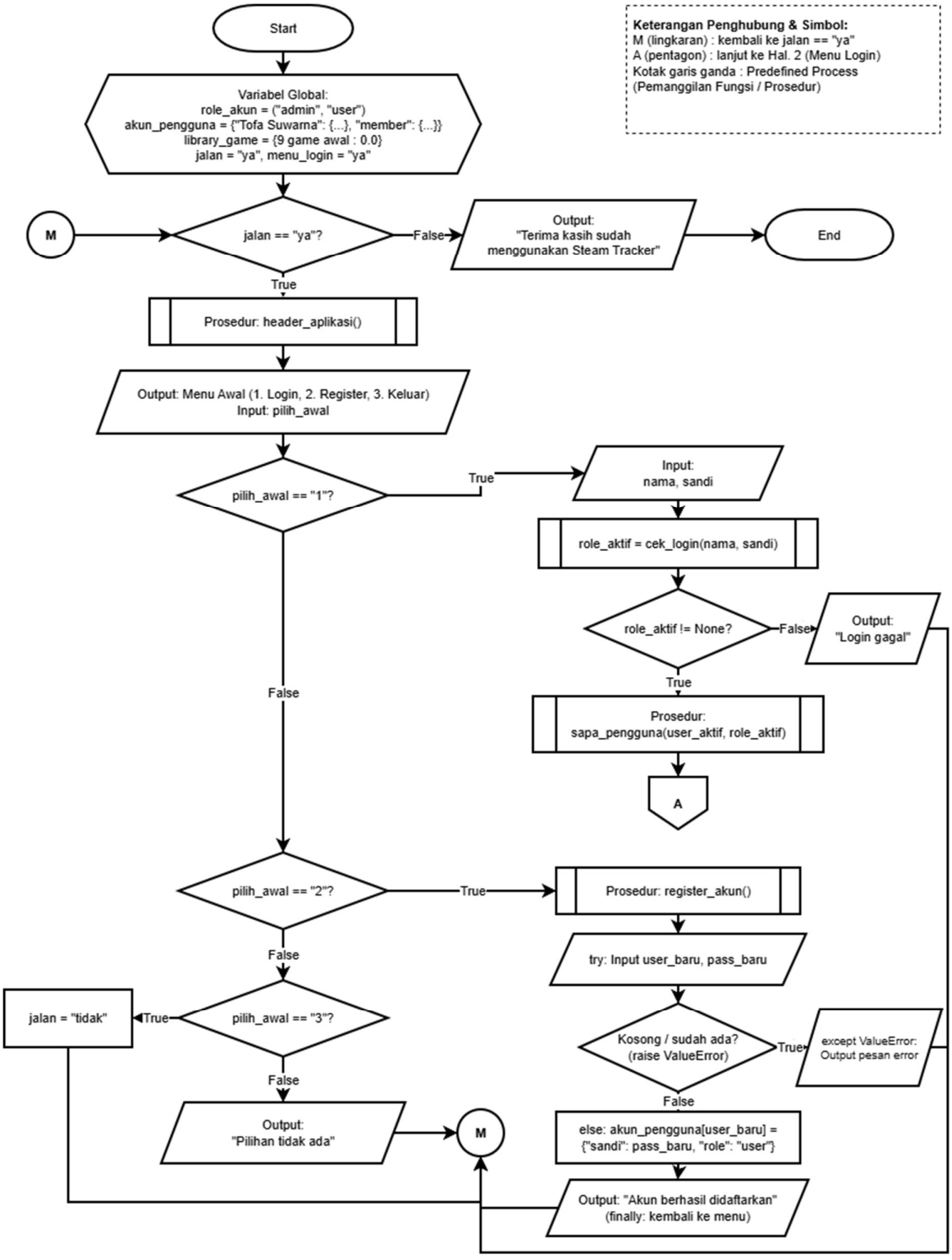
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

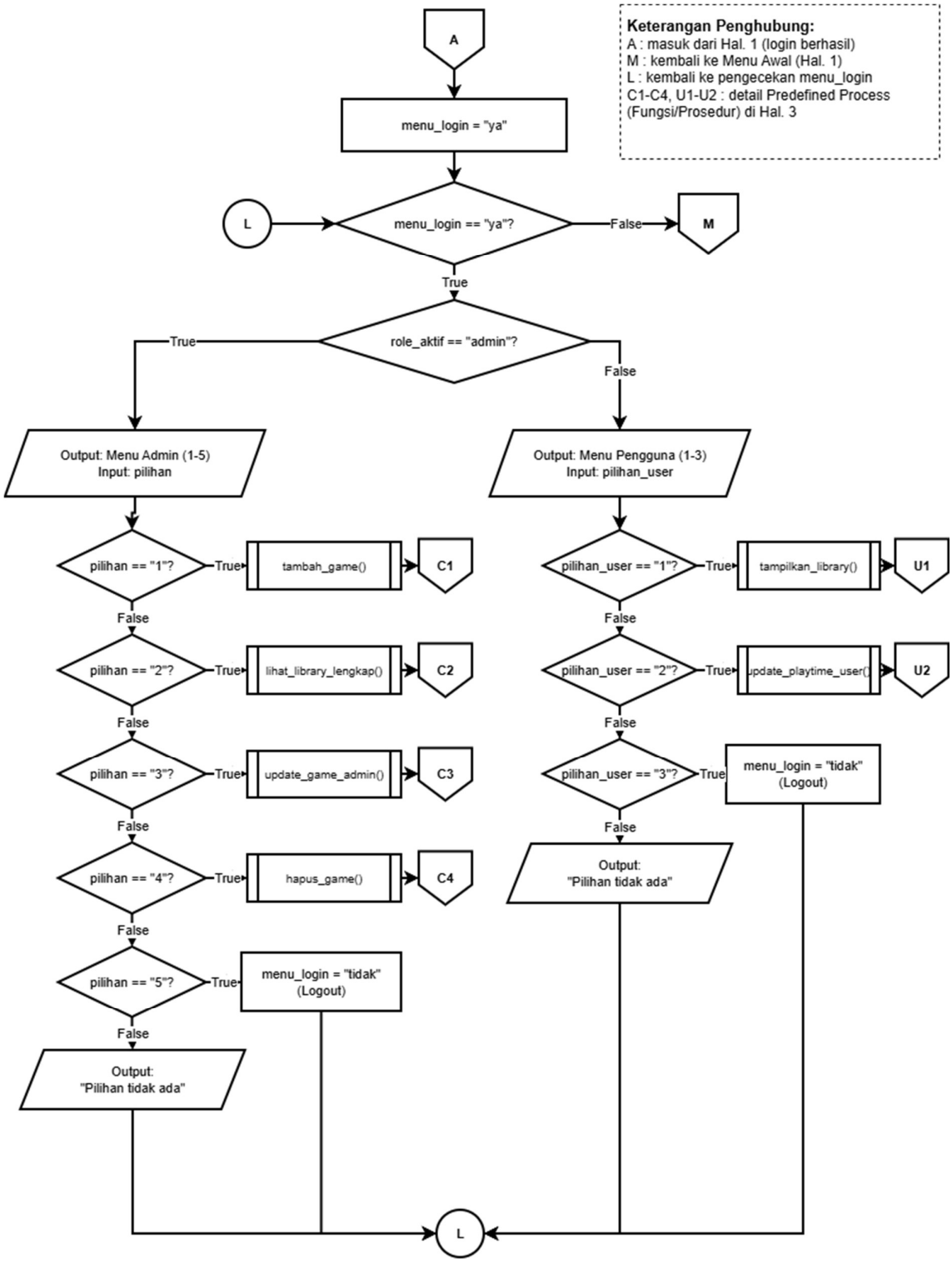
SAMARINDA

2026

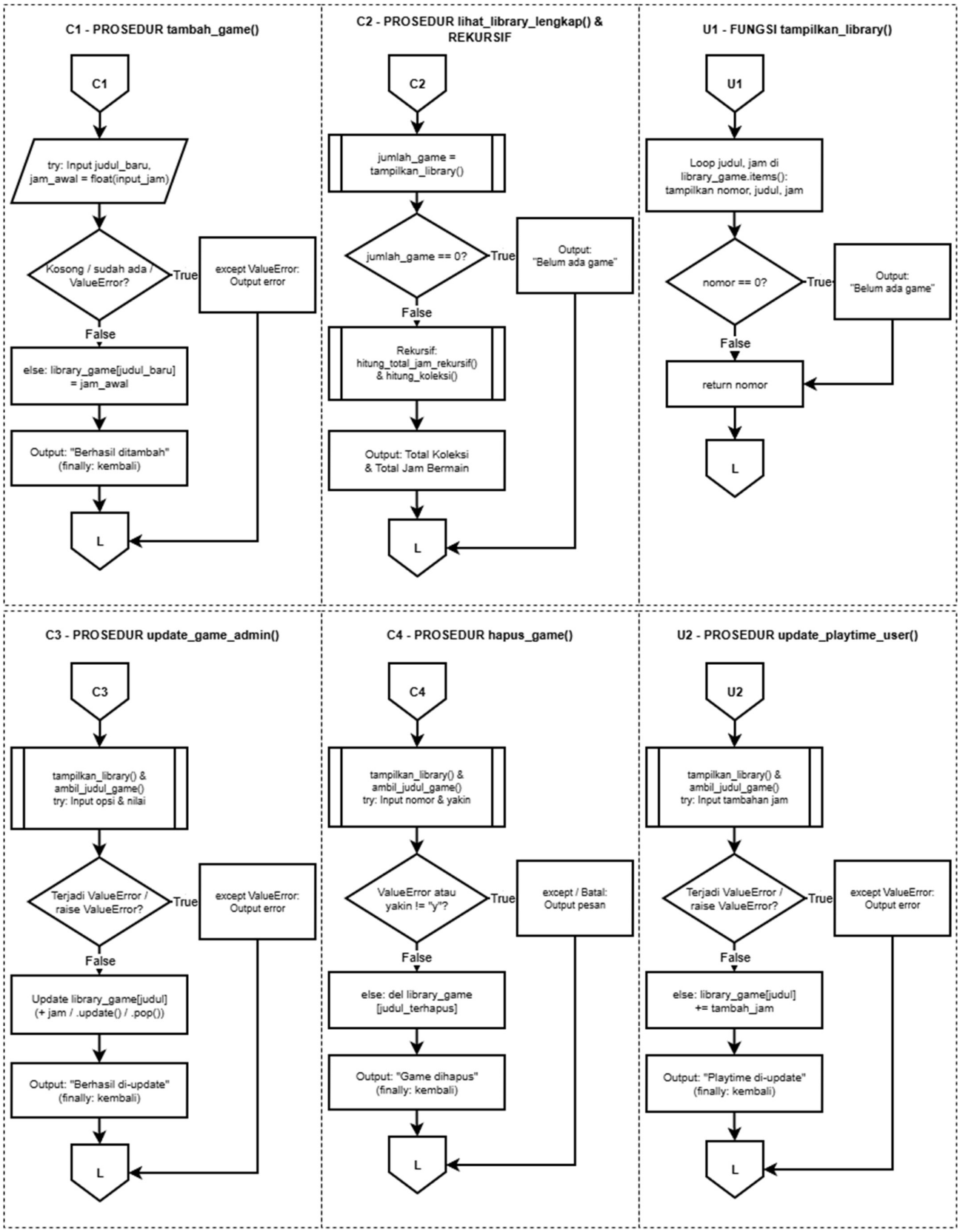
1. Flowchart



Gambar 1.1 Flowchart Alur



Gambar 1.2 Flowchart Menu Login



Gambar 1.3 Flowchart Fitur

Program diawali dengan menginisialisasi lima variabel global di luar fungsi, yaitu tuple status akses akun admin dan user, nested dictionary akun pengguna, dictionary library game yang menyimpan sembilan judul game awal beserta jam mainnya, serta variabel pengontrol perulangan utama dan menu login. Saat perulangan utama berjalan, program memanggil prosedur tanpa parameter untuk membersihkan layar dan menampilkan header aplikasi, lalu menyajikan menu awal yang berisi opsi Login, Register, dan Keluar. Apabila pengguna memilih opsi Register, program akan menjalankan prosedur registrasi yang dilengkapi blok penanganan error try-except-else-finally serta raise ValueError untuk memastikan nama pengguna dan kata sandi baru tidak kosong maupun duplikat sebelum disimpan ke dalam sistem.

Jika pengguna memilih opsi Login, program akan menerima masukan nama serta sandi, lalu memanggil fungsi berparameter untuk mencocokkan data tersebut dengan dictionary akun pengguna dan mengembalikan status rolenya. Apabila data tidak cocok, fungsi mengembalikan nilai kosong dan menampilkan peringatan gagal login. Sebaliknya, jika login berhasil, program memanggil prosedur berparameter untuk menyapa pengguna beserta status rolenya, kemudian membawanya masuk ke perulangan halaman menu utama sesuai hak akses masing-masing.

Bagi pengguna berstatus Admin, tersedia lima pilihan menu yang masing-masing dieksekusi melalui pemanggilan prosedur terpisah. Menu pertama memanggil prosedur tambah game baru yang memvalidasi judul dan angka jam bermain menggunakan try-except dan raise ValueError. Menu kedua memanggil prosedur lihat library lengkap yang di dalamnya memanggil fungsi tanpa parameter untuk menampilkan daftar game dan menghitung jumlah koleksi, sekaligus memanggil fungsi rekursif untuk menjumlahkan seluruh waktu bermain game secara bertahap hingga mencapai kondisi berhenti atau base case. Menu ketiga dan keempat memanggil prosedur update serta hapus game yang memanfaatkan fungsi berparameter pencari judul game berdasarkan nomor urut dan dilindungi oleh blok try-except agar program tidak berhenti mendadak jika pengguna salah memasukkan huruf atau nomor di luar daftar.

Sementara itu, bagi Pengguna Biasa, akses menu dibatasi hanya untuk melihat daftar library game melalui fungsi tampilkan library dan menambah jam bermain melalui prosedur update playtime user yang juga telah dilindungi blok try-except-else-finally. Setiap kali sebuah prosedur selesai dijalankan, blok finally akan meminta pengguna menekan Enter untuk kembali ke menu utama hingga mereka memilih opsi Logout untuk kembali ke halaman awal, dan program akan berhenti sepenuhnya pada simbol End ketika pengguna memilih opsi Keluar.

2. Deskripsi Singkat Program

Program ini merupakan aplikasi manajemen koleksi game berbasis terminal bernama "Sistem Pencatatan dan Tracking Playtime Game Steam". Tujuan utama dari program ini adalah untuk mencatat daftar game di dalam library Steam (yang diawali dengan 9 koleksi game seperti seri Dark Souls, Elden Ring, Nightreign, Resident Evil, dan Persona) sekaligus melakukan tracking berapa lama playtime pada setiap game dalam satuan jam.

Pada Posttest 7 ini, kode program dari Posttest 6 telah dimodularisasi menggunakan Fungsi (dengan parameter dan tanpa parameter menggunakan return), Prosedur (def tanpa return), Fungsi Rekursif (hitung_total_jam_rekursif) untuk mengakumulasikan total jam bermain, serta pemisahan Variabel Global dan Variabel Lokal sesuai Modul 7.1. Selain itu, validasi input pada seluruh fitur CRUD dan sistem Multiuser (Admin dan Pengguna Biasa) telah diperbarui menggunakan mekanisme Error Handling sesuai Modul 7.2 yang mencakup blok try, except ValueError, else, finally, serta pembuatan pesan error kustom menggunakan keyword raise ValueError.

3. Source Code

A. Fitur Login & Register

Fitur ini digunakan untuk menyimpan data role menggunakan tuple, menyimpan data akun menggunakan nested dictionary dan koleksi game menggunakan dictionary, serta memverifikasi proses Login multiuser menggunakan fungsi dan prosedur.

```
import os

role_akun = ("admin", "user")

akun_pengguna = {
    "Tofa Suwarna": {
        "sandi": "037",
        "role": role_akun[0]
    },
    "member": {
        "sandi": "member123",
        "role": role_akun[1]
    }
}

library_game = {
    "Dark Souls Remastered": 0.0,
    "Dark Souls II": 0.0,
    "Dark Souls III": 0.0,
    "Elden Ring": 0.0,
    "Nightreign": 0.0,
    "RE4 Remake": 0.0,
    "RE Requiem": 0.0,
    "Persona 3": 0.0,
    "Persona 5 Royal": 0.0
}

jalan = "ya"
menu_login = "ya"

def header_aplikasi():
    os.system("cls")
    print("=======================================================")
    print("----- ^-^ SELAMAT DATANG DI STEAM TRACKER ^-^ -----")
    print("   Sistem Pencatatan & Tracking Playtime Game Steam  ")
    print("=======================================================")

def sapa_pengguna(nama_user, status_role):
    os.system("cls")
    print("Yeayy! Login Berhasil Cuy, selamat datang " + nama_user)
```

```
    print("Status Role Anda : " + status_role)
    input("\nTekan Enter untuk masuk ke menu utama...")

def cek_login(nama_input, sandi_input):
    for user_key, data_val in akun_pengguna.items():
        if user_key == nama_input and data_val["sandi"] == sandi_input:
            return data_val["role"]
    return None
```

B. Fitur Tambah Game Baru (Create)

Fitur ini digunakan oleh Admin prosedur tambah_game() untuk menambahkan data game baru beserta playtime awalnya ke dalam dictionary library_game. Dilengkapi error handling try-except dan raise ValueError untuk memastikan judul belum terdaftar dan input jam berupa angka yang valid.

```
def tambah_game():
    os.system("cls")
    print("--- TAMBAH GAME BARU KE LIBRARY ---")
    try:
        judul_baru = input("Masukkan Judul Game : ")
        if judul_baru == "":
            raise ValueError("\n[!] Waduh, judul game nggak boleh kosong wak!")
        if library_game.get(judul_baru) != None:
            raise ValueError("\n[!] Game ini sudah ada di dalam library wak!")

        input_jam = input("Masukkan Playtime Awal (Jam, contoh 0 atau 6.7): ")
        jam_awal = float(input_jam)
        if jam_awal < 0:
            raise ValueError("\n[!] Jam bermain tidak boleh bernilai negatif
wak!")
    except ValueError as e:
        if "could not convert" in str(e):
            print("\n[!] Input jam bermain tidak valid! Harus berupa angka.")
        else:
            print(e)
    else:
        library_game[judul_baru] = jam_awal
        print(f"\n[+] Mantap! Berhasil menambahkan -> {judul_baru}: {jam_awal}
Jam")
    finally:
        input("\nTekan Enter untuk kembali ke menu...")
```

C. Fitur Lihat Daftar Library Game (Read)

Fitur ini berfungsi untuk membaca dan menampilkan seluruh daftar koleksi game di dalam dictionary library_game menggunakan fungsi tampilkan_library(), menghitung jumlah koleksi dengan hitung_koleksi(), serta menghitung total akumulasi jam bermain menggunakan Fungsi Rekursif hitung_total_jam_rekursif(daftar_jam, indeks).

```
def hitung_koleksi():
    jumlah = len(library_game)
    return jumlah

def tampilkan_library():
    print("--- DAFTAR LIBRARY GAME ---")
    nomor = 0
    for judul, jam in library_game.items():
        nomor += 1
        print(f"{nomor}. {judul}: {jam} Jam")

    if nomor == 0:
        print("Belum ada game di dalam library.")
    return nomor

def hitung_total_jam_rekursif(daftar_jam, indeks):
    if indeks == len(daftar_jam):
        return 0.0
    else:
        return daftar_jam[indeks] + hitung_total_jam_rekursif(daftar_jam, indeks
+ 1)

def lihat_library_lengkap():
    os.system("cls")
    jumlah_game = tampilkan_library()
    if jumlah_game > 0:
        list_jam = list(library_game.values())
        total_jam = hitung_total_jam_rekursif(list_jam, 0)
        print("----------------------------")
        print(f"Total Koleksi Game : {hitung_koleksi()} Game")
        print(f"Total Jam Bermain  : {total_jam} Jam")
    input("\nTekan Enter untuk kembali ke menu...")
```

D. Fitur Update / Tracking Playtime Game (Update)

Fitur ini digunakan untuk memperbarui total jam bermain maupun mengubah judul game pada dictionary library_game dengan memanfaatkan fungsi berparameter ambil_judul_game (nomor_target) dan prosedur update yang dilengkapi try-except.

```
def ambil_judul_game(nomor_target):
    urutan = 0
    for judul in library_game.keys():
        urutan += 1
        if urutan == nomor_target:
            return judul
    return None

def update_playtime_user():
    os.system("cls")
    print("--- TRACKING PLAYTIME GAME STEAM ---")
    jumlah_game = tampilkan_library()

    if jumlah_game > 0:
        print("----------------------------")
        try:
            nomor_target = int(input("Masukkan nomor game yang baru dimainkan:
"))
            if nomor_target < 1 or nomor_target > jumlah_game:
                raise ValueError("\n[!] Nomor game tidak ditemukan di daftar!")

            judul_terpilih = ambil_judul_game(nomor_target)
            tambah_jam = float(input(f"Masukkan tambahan jam main untuk
'{judul_terpilih}': "))
            if tambah_jam < 0:
                raise ValueError("\n[!] Tambahan jam main tidak boleh negatif!")
        except ValueError as e:
            if "invalid literal" in str(e) or "could not convert" in str(e):
                print("\n[!] Input tidak valid! Harus berupa angka.")
            else:
                print(e)
        else:
            library_game[judul_terpilih] += tambah_jam
            print(f"\n[+] Mantap! Playtime sekarang -> {judul_terpilih}:
{library_game[judul_terpilih]} Jam")
        finally:
            input("\nTekan Enter untuk kembali ke menu...")
    else:
        input("\nTekan Enter untuk kembali ke menu...")
```

E. Fitur Hapus Game dari Library (Delete)

Fitur ini digunakan oleh Admin melalui prosedur hapus_game() untuk menghapus salah satu game dari dictionary library_game berdasarkan nomor urut yang dipilih menggunakan perintah del.

```
def hapus_game():
    os.system("cls")
    print("--- HAPUS GAME DARI LIBRARY ---")
    jumlah_game = tampilkan_library()

    if jumlah_game > 0:
        print("---------------------------")
        try:
            nomor_hapus = int(input("Masukkan nomor urut game yang ingin
dihapus: "))
            if nomor_hapus < 1 or nomor_hapus > jumlah_game:
                raise ValueError("\n[!] Nomor urut game tidak ada di dalam
daftar!")
        except ValueError as e:
            if "invalid literal" in str(e):
                print("\n[!] Input harus berupa angka nomor urut!")
            else:
                print(e)
        else:
            judul_terhapus = ambil_judul_game(nomor_hapus)
            yakin = input(f"Yakin mau menghapus '{judul_terhapus}'? (y/n): ")
            if yakin == "y" or yakin == "Y":
                del library_game[judul_terhapus]
                print(f"\n[-] Game '{judul_terhapus}' berhasil dihapus dari
library!")
            else:
                print("\n[i] Penghapusan game dibatalkan.")
        finally:
            input("\nTekan Enter untuk kembali ke menu...")
    else:
        input("\nTekan Enter untuk kembali ke menu...")
```

F. Fitur Logout dan Keluar Program

Fitur ini bertugas untuk mengakhiri sesi pengguna yang sedang login (menu_login = "tidak") agar kembali ke halaman menu awal, serta menghentikan jalannya perulangan utama program (jalan = "tidak") ketika pengguna memilih keluar dari aplikasi.

```
elif pilihan == "5":
                        menu_login = "tidak"
                        print("\nBerhasil logout dari akun Atmint.")
                        input("Tekan Enter untuk kembali ke halaman awal...")

    elif pilih_awal == "3":
        os.system("cls")
        print("=====================================================")
        print("  Terima kasih sudah menggunakan Steam Tracker Cik!  ")
```

```
        print("=====================================================")
        jalan = "tidak"
```

G. Fitur Error Handling

Fitur ini menerapkan blok try, except ValueError, else, finally, serta pembuatan error kustom menggunakan raise ValueError sesuai Modul 7.2 pada proses input data untuk menangani kesalahan input tipe data angka (int/float) maupun kondisi input kosong atau di luar jangkauan daftar.

```
def register_akun():
    os.system("cls")
    print("----- ^-^ REGISTER AKUN BARU ^-^ -----")
    try:
        user_baru = input("Buat Username Baru : ")
        pass_baru = input("Buat Password Baru : ")

        if user_baru == "" or pass_baru == "":
            raise ValueError("\n[!] Username dan Password tidak boleh kosong!")
        if akun_pengguna.get(user_baru) != None:
            raise ValueError("\n[!] Waduh, Username tersebut sudah terdaftar!
Coba nama lain.")
    except ValueError as e:
        print(e)
    else:
        akun_pengguna[user_baru] = {
            "sandi": pass_baru,
            "role": role_akun[1]
        }
        print(f"\n[+] Yeayy! Akun '{user_baru}' berhasil didaftarkan sebagai
Pengguna Biasa!")
    finally:
        input("\nTekan Enter untuk kembali ke menu awal...")
```

H. Penerapan Masing-masing 2 Fungsi dengan Parameter dan Tanpa Parameter

Sesuai ketentuan posttest 7 kali ini, fungsi ditandai dengan penggunaan def dan wajib mengembalikan nilai (return). Pada program ini terdapat 2 fungsi tanpa parameter (hitung_koleksi dan tampilkan_library) serta 2 fungsi dengan parameter (cek_login dan ambil_judul_game).

```
#2 Fungsi Tanpa Parameter
def hitung_koleksi():
    jumlah = len(library_game)
    return jumlah

def tampilkan_library():
    print("--- DAFTAR LIBRARY GAME ---")
    nomor = 0
    for judul, jam in library_game.items():
        nomor += 1
        print(f"{nomor}. {judul}: {jam} Jam")

    if nomor == 0:
        print("Belum ada game di dalam library.")
    return nomor


#2 Fungsi Dengan Parameter
def cek_login(nama_input, sandi_input):
    for user_key, data_val in akun_pengguna.items():
        if user_key == nama_input and data_val["sandi"] == sandi_input:
            return data_val["role"]
    return None

def ambil_judul_game(nomor_target):
    urutan = 0
    for judul in library_game.keys():
        urutan += 1
        if urutan == nomor_target:
            return judul
    return None
```

I. Penerapan 2 Prosedur

Berbeda dengan fungsi, prosedur menggunakan def untuk memecah sub-program tetapi tidak mengembalikan nilai (tidak memakai return). Program ini menggunakan banyak prosedur (seperti fitur Create, Read, Update, Delete), contohnya adalah prosedur tanpa parameter (header_aplikasi) dan prosedur dengan parameter (sapa_pengguna).

```
#1. Prosedur Tanpa Parameter
def header_aplikasi():
    os.system("cls")
    print("=======================================================")
    print("----- ^-^ SELAMAT DATANG DI STEAM TRACKER ^-^ -----")
    print("   Sistem Pencatatan & Tracking Playtime Game Steam  ")
    print("=======================================================")
```

```
#2. Prosedur Dengan Parameter
def sapa_pengguna(nama_user, status_role):
    os.system("cls")
    print("Yeayy! Login Berhasil Cuy, selamat datang " + nama_user)
    print("Status Role Anda : " + status_role)
    input("\nTekan Enter untuk masuk ke menu utama...")
```

J. Penggunaan Minimal 3 Variabel Global dan 5 Variabel Lokal

Variabel global dideklarasikan di luar fungsi sehingga dapat diakses di seluruh cakupan program. Sementara itu, variabel lokal dideklarasikan di dalam blok fungsi atau prosedur sehingga hanya bisa diakses dalam cakupan fungsi tersebut berada. kode di bawah ini contoh dari variabel variabel berikut, ditampilkan 3 variabel global utama (role_akun, akun_pengguna, library_game) serta 5 variabel lokal yang tersebar di beberapa fungsi dan prosedur (jumlah, nomor, user_baru, pass_baru, dan judul_baru).

```
#VARIABEL GLOBAL
role_akun = ("admin", "user")

akun_pengguna = {
    "Tofa Suwarna": {"sandi": "037", "role": role_akun[0]},
    "member": {"sandi": "member123", "role": role_akun[1]}
}

library_game = {
    "Dark Souls Remastered": 0.0,
    "Dark Souls II": 0.0,
    "Dark Souls III": 0.0,
    "Elden Ring": 0.0,
    "Nightreign": 0.0,
    "RE4 Remake": 0.0,
    "RE Requiem": 0.0,
    "Persona 3": 0.0,
    "Persona 5 Royal": 0.0
}

#VARIABEL LOKAL
def hitung_koleksi():
    jumlah = len(library_game)
    return jumlah

def tampilkan_library():
    print("--- DAFTAR LIBRARY GAME ---")
    nomor = 0
```

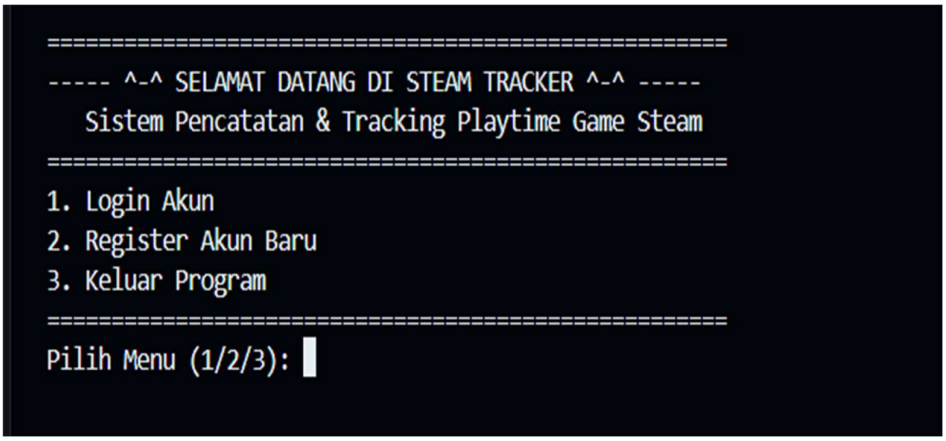
```
    for judul, jam in library_game.items():
        nomor += 1
        print(f"{nomor}. {judul}: {jam} Jam")
    if nomor == 0:
        print("Belum ada game di dalam library.")
    return nomor

def register_akun():
    os.system("cls")
    print("----- ^-^ REGISTER AKUN BARU ^-^ -----")
    try:
        user_baru = input("Buat Username Baru : ")
        pass_baru = input("Buat Password Baru : ")
        if user_baru == "" or pass_baru == "":
            raise ValueError("\n[!] Username dan Password tidak boleh kosong!")
        if akun_pengguna.get(user_baru) != None:
            raise ValueError("\n[!] Waduh, Username tersebut sudah terdaftar!
Coba nama lain.")
    except ValueError as e:
        print(e)
    else:
        akun_pengguna[user_baru] = {
            "sandi": pass_baru,
            "role": role_akun[1]
        }
        print(f"\n[+] Yeayy! Akun '{user_baru}' berhasil didaftarkan sebagai
Pengguna Biasa!")
    finally:
        input("\nTekan Enter untuk kembali ke menu awal...")

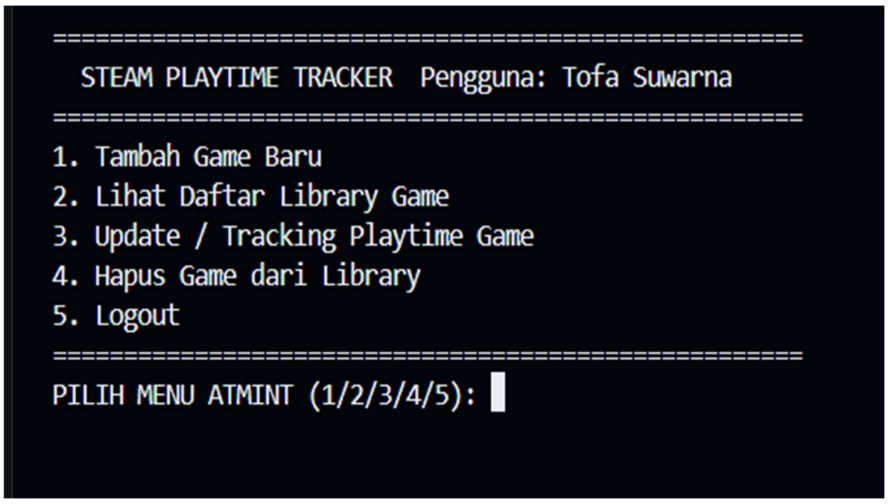
def tambah_game():
    os.system("cls")
    print("--- TAMBAH GAME BARU KE LIBRARY ---")
    try:
        judul_baru = input("Masukkan Judul Game : ")
        if judul_baru == "":
            raise ValueError("\n[!] Waduh, judul game nggak boleh kosong wak!")
        if library_game.get(judul_baru) != None:
            raise ValueError("\n[!] Game ini sudah ada di dalam library wak!")
        input_jam = input("Masukkan Playtime Awal (Jam, contoh 0 atau 6.7): ")
        jam_awal = float(input_jam)
        if jam_awal < 0:
            raise ValueError("\n[!] Jam bermain tidak boleh bernilai negatif
wak!")
    except ValueError as e:
        if "could not convert" in str(e):
            print("\n[!] Input jam bermain tidak valid! Harus berupa angka.")
        else:
            print(e)
    else:
        library_game[judul_baru] = jam_awal
```

```
        print(f"\n[+] Mantap! Berhasil menambahkan -> {judul_baru}: {jam_awal}
Jam")
    finally:
        input("\nTekan Enter untuk kembali ke menu...")
```

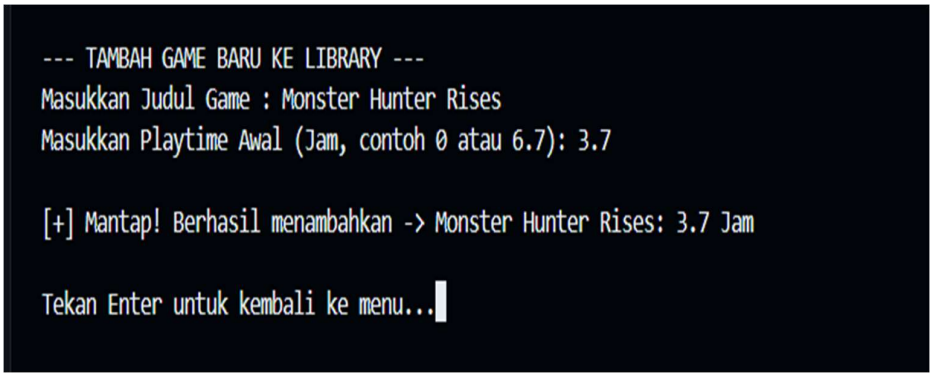
4. Hasil Output



Gambar 4.1 Output Menu Awal



Gambar 4.2 Output Login Telah Berhasil sebagai Admin



Gambar 4.3 Output (1) Tambah Game baru ke Library (Create)

```
--- DAFTAR LIBRARY GAME ---
1. Dark Souls Remastered: 0.0 Jam
2. Dark Souls II: 0.0 Jam
3. Dark Souls III: 0.0 Jam
4. Elden Ring: 0.0 Jam
5. Nightreign: 0.0 Jam
6. RE4 Remake: 0.0 Jam
7. RE Requiem: 0.0 Jam
8. Persona 3: 0.0 Jam
9. Persona 5 Royal: 0.0 Jam
10. Monster Hunter Rises: 3.7 Jam
--------------------------
Total Koleksi Game : 10 Game
Total Jam Bermain  : 3.7 Jam

Tekan Enter untuk kembali ke menu...
```

Gambar 4.4 Output (2) Melihat Daftar Library Game (Read)

```
--- UPDATE & TRACKING PLAYTIME GAME ---
--- DAFTAR LIBRARY GAME ---
1. Dark Souls Remastered: 0.0 Jam
2. Dark Souls II: 0.0 Jam
3. Dark Souls III: 0.0 Jam
4. Elden Ring: 0.0 Jam
5. Nightreign: 0.0 Jam
6. RE4 Remake: 0.0 Jam
7. RE Requiem: 0.0 Jam
8. Persona 3: 0.0 Jam
9. Persona 5 Royal: 0.0 Jam
10. Monster Hunter Rises: 3.7 Jam
--------------------------
Masukkan nomor urut game yang mau di-update: 4

Game Terpilih : Elden Ring: 0.0 Jam
1. Tambah Jam Bermain
2. Ganti Total Jam Bermain Baru
3. Ganti Judul Game
Pilih opsi update (1/2/3): 1
Masukkan tambahan jam bermain (contoh 6.7): 119.8

[+] Playtime berhasil di-update -> Elden Ring: 119.8 Jam

Tekan Enter untuk kembali ke menu...
```

Gambar 4.5 Output (3) Mengupdate & Tracking Playtime Game (Update)

```
--- HAPUS GAME DARI LIBRARY ---
--- DAFTAR LIBRARY GAME ---
1. Dark Souls Remastered: 0.0 Jam
2. Dark Souls II: 0.0 Jam
3. Dark Souls III: 0.0 Jam
4. Elden Ring: 119.8 Jam
5. Nightreign: 0.0 Jam
6. RE4 Remake: 0.0 Jam
7. RE Requiem: 0.0 Jam
8. Persona 3: 0.0 Jam
9. Persona 5 Royal: 0.0 Jam
10. Monster Hunter Rises: 3.7 Jam
--------------------------
Masukkan nomor urut game yang ingin dihapus: 4
Yakin mau menghapus 'Elden Ring'? (y/n): y

[-] Game 'Elden Ring' berhasil dihapus dari library!

Tekan Enter untuk kembali ke menu...
```

Gambar 4.6 Output (4) Menghapus Game dari Library (Delete)

```
----- ^-^ REGISTER AKUN BARU ^-^ -----
Buat Username Baru : ilham
Buat Password Baru : ilham123

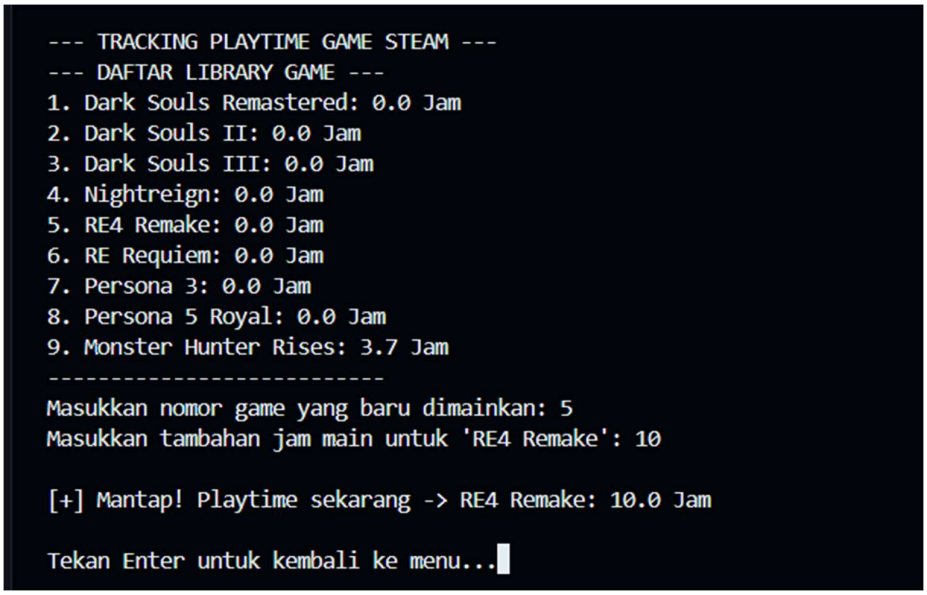
[+] Yeayy! Akun 'ilham' berhasil didaftarkan sebagai Pengguna Biasa!

Tekan Enter untuk kembali ke menu awal...
```

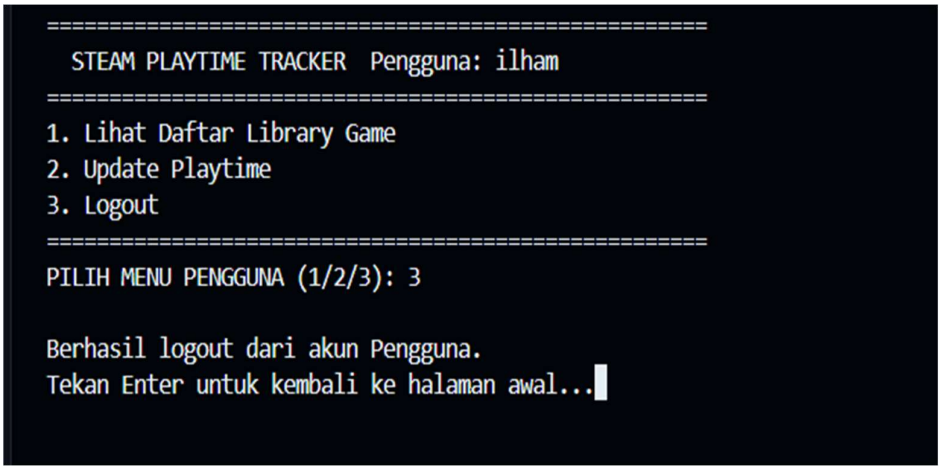
Gambar 4.7 Output Registrasi Akun User berhasil

```
==================================================
  STEAM PLAYTIME TRACKER  Pengguna: ilham
==================================================
1. Lihat Daftar Library Game
2. Update Playtime
3. Logout
==================================================
PILIH MENU PENGGUNA (1/2/3):
```

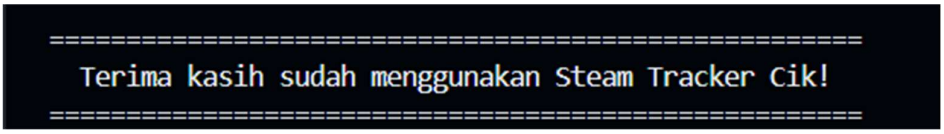
Gambar 4.8 Output Menu dari Program sebagai User



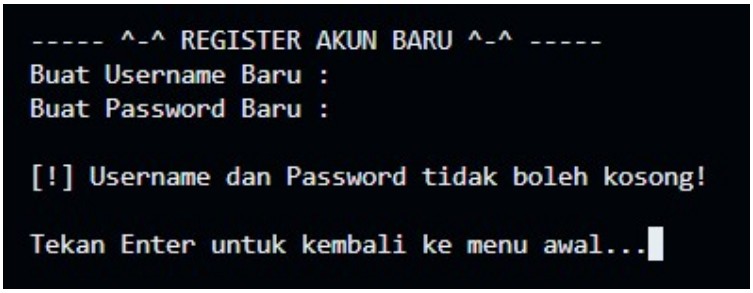
Gambar 4.9 Output Tracking Playtime Game Steam sebagai User/member



Gambar 4.10 Output Logout



Gambar 4.11 Output ketika Keluar Program pada Menu Awal



Gambar 4.12 Output Error Handling

5. Langkah-langkah Git

5. 1 Git Add

```
PS D:\Code\Py\praktikum-apd-2026> git add .
PS D:\Code\Py\praktikum-apd-2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   posttest/posttest-apd-7/037-Tofa Suwarna-PT-7.py
```

Gambar 5.1 Git Add

Perintah git add . digunakan untuk menambahkan seluruh file baru maupun perubahan file yang ada supaya dicatat oleh Git.

5.2 Git Commit

```
PS D:\Code\Py\praktikum-apd-2026> git commit -m "committ"
[main 8b1c220] committ
 1 file changed, 344 insertions(+)
 create mode 100644 posttest/posttest-apd-7/037-Tofa Suwarna-PT-7.py
```

Gambar 5.2 Git Commit

Perintah git commit digunakan untuk merekam riwayat perubahan file ke dalam repositori lokal. (-m) digunakan sebagai deskripsi singkat mengenai perubahan yang telah dilakukan atau bisa diisi dengan bebas juga.

5.3 Git Push

```
PS D:\Code\Py\praktikum-apd-2026> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 3.33 KiB | 1.67 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/tofsuw23/praktikum-apd-2026.git
   f137d1b..8b1c220  main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3 Git Push

Perintah git push digunakan untuk mengupload dan menyinkronkan semua commit atau file dari yang tadinya hanya berada pada komputer (lokal) ke repositori remote di GitHub. Jika berhasil maka outputnya sama seperti pada gambar diatas.